Medical - Clinical - Research
Research Article
ISSN: 3069-9088

SiPy Kernel for Jupyter

Wira Bin Ambel[1] and Maurice HT Ling[2,3]*

[1]School of Health & Life Sciences, Teesside University, UK.

[2]Newcastle Australia Institute of Higher Education, University of Newcastle, Australia.

[3]HOHY PTE LTD, Singapore.

Abstract

Jupyter Notebook and JupyterLab have become widely adopted platforms for interactive data analysis, teaching, and reproducible research. While many polyglot kernels enable multi-language execution through in-notebook language switching and in-memory data exchange; such approaches often blur language boundaries, resulting in increased coupling to kernel-specific abstractions, and complicate reproducibility outside the notebook environment. We present the SiPy kernel, a lightweight Jupyter kernel designed not as a multi-language interpreter but as a coordination interface for script-based scientific workflows. SiPy treats R, Julia, Python, and shell scripts as first-class external artefacts, executing them as standalone scripts rather than embedded notebook fragments. This design preserves explicit execution boundaries, enhances portability, and ensures that analytical components remain runnable outside Jupyter. The kernel integrates the SiPy interactive shell into Jupyter while maintaining minimal dependence on internal notebook APIs and avoiding interpreter embedding. Features include controlled execution with configurable timeouts, structured output capture, and support for HTML rendering within cells. By positioning Jupyter as a narrative and documentation layer rather than a computational substrate, the SiPy kernel offers an alternative architectural model for polyglot scientific computing that prioritizes transparency, reproducibility, and long-term maintainability over convenience-driven abstraction.

Keywords

Jupyter kernel, Reproducible research, Polyglot computing, Script-based workflow, Computational provenance, Scientific workflow orchestration.

Introduction

Jupyter Notebook is a spinoff from IPython [1,2] in 2014, and with JupyterLab as the next-generation enhancement, it has emerged as a de facto platform for data analytics and scientific computation [3] – widely adopted across data science and research workflows [4]. Its notebook-based model; integrating executable code with narrative text, equations, and visual output; has been effective for exploratory and analytical storytelling [5], and a pedagogical medium for teaching programming and data analysis [6,7]. Studies report improved student engagement and learning outcomes when Jupyter Notebooks are used as the primary instructional interface [8-11]. It also provide a natural mechanism for workflow documentation by recording intermediate results, execution order, and analytical intent within a single artefact [12]. This structure plays a central role in contemporary reproducible research practices by making computational processes more transparent and auditable [12-14], even though reproducibility is not guaranteed by notebooks alone [15,16]. Nevertheless, it is a familiar user interface for data analysts and scientists [17], independent of any specific computational framework, and well suited as a front-end for domain-specific kernels.

SiPy [18] is a lightweight statistical interface written in Python, and has been demonstrated as a potential platform for incorporating R methods while reducing the learning curve needed to learn R [19]; as well as language-agnostic system for executing external scripts, emphasizing transparency, provenance, and minimal abstraction over underlying tools. Against this backdrop, we present a Jupyter kernel for SiPy, which is motivated not by the introduction of a new computational language but by the use of Jupyter as a coordination and documentation interface for script-based scientific workflows [20]. Integrating SiPy with Jupyter retains the familiarity and communicative strengths of the notebook interface while avoiding tight coupling between analytical logic and the notebook execution model. Hence, Jupyter functions primarily as an interactive front-end and narrative layer, while SiPy serves as an execution coordinator invoking self-contained scripts that remain runnable outside the notebook environment.

Implementation

The SiPy Kernel is implemented as a custom Jupyter kernel that integrates the SiPy interactive shell into both Jupyter Notebook and JupyterLab [21] while preserving SiPy's philosophy of explicit, script-level execution. Design choices emphasize explicit execution boundaries and minimal coupling with the notebook environment. Unlike kernels that embed interpreters or rely on in-process bindings, the SiPy Kernel treats external scripts and language back ends as first-class invocation targets. This reduces dependence on fragile internal APIs, enhances robustness to changes in external runtimes, and makes analytical workflows executable both inside and outside the

Correspondence to: Maurice HT Ling, Newcastle Australia Institute of Higher Education, University of Newcastle, Australia, and HOHY PTE LTD, Singapore.

Received: 11 Feb 2026; **Accepted:** 17 Mar 2026; **Published:** 25 Mar 2026

Citation: Wira Bin Ambel, Maurice HT Ling. SiPy Kernel for Jupyter. Med - Clin - Res. 2026;2(3):59-65. DOI: https://www.doi.org/10.52106/3069-9088.1021.

Copyright
© 2026 Authors. This is an open- access article distributed under the terms of the Creative Commons Attribution 4.0 International license.

notebook. The kernel is intentionally lightweight; it does not embed additional language interpreters nor emulate language semantics. Instead, it treats Jupyter as an interactive interface and documentation layer on top of SiPy's existing orchestration framework.

At launch, the kernel locates the core SiPy script (sipy.py) either through the SIPY_PY environment variable or by walking up from the current working directory. Once identified, the containing directory is added to Python's module search path and becomes the kernel's working directory, ensuring consistent script resolution across notebook cells. The kernel then dynamically loads the SiPy module using Python's import mechanisms and instantiates a SiPy_Shell object, which handles command interpretation. Execution of user input is coordinated through the kernel's do_execute() method. Cells are treated as sequences of SiPy commands rather than statements in a general-purpose language. Empty lines and comment lines are ignored. Actual commands are passed line by line to SiPy_Shell. interpret(). To prevent blocking the Jupyter server, each cell's execution occurs in a separate worker thread with a configurable timeout. If execution exceeds the allotted time, it terminates with a timeout error. The kernel captures standard output and standard error generated during execution and returns them to the front end. To improve readability, multi-line output is rendered as rich display data (e.g., Markdown) when appropriate, while single-line output remains plain text. An explicit %%html prefix supports user-driven HTML rendering, enabling richer narrative elements within notebooks.

Overall, the implementation positions the SiPy Kernel as an execution coordinator rather than a language interpreter: Jupyter provides the interactive and narrative surface, while SiPy manages command interpretation, external script invocation, and output capture in a transparent and reproducible manner. This architecture supports integration with R, Julia, Python, and shell scripts as external computational back ends, aligns with reproducible research practices, and preserves the familiar, document-centric experience of the notebook.

Testing and Usage

The command to launch SiPy kernel in Windows is "python sipy.py jupyter", which will trigger the installation of the SiPy kernel into the Python environment before launching JupyterLab. This method, though takes several seconds longer due to the installation process, ensures that an updated version of SiPy kernel is always present. For Mac OSX, "python sipy.py jupyter" command will only install SiPy kernel but may not launch JupyterLab; hence, a follow-up command "jupyterlab" may be required.

A testing notebook, called jupyter_notebook (https://bit.ly/SiPy_Notebook_Sample) is found in sipy/notebook folder, and shown in Figure 1. This notebook tests the connectivity and functionality of SiPy kernel to be used a user interface to SiPy shell. In Figure 1A, plain text is rendered in HTML using %%html operator, an Excel file is being read into SiPy, changing of timeout and security parameters, followed by a attempt to execute an unsafe command which is successfully intercepted. Figure 1B demonstrates the successful execution of t-tests, and R-based ANOVA; thereby, suggesting that connection between SiPy kernel and SiPy itself is established. Figure 1C shows the execution of commands and Figure 1D shows execution of Julia script, and Python script – both functions are key features added in SiPy 0.8.0 [22].

Next, we analysed as well-known diabetes data set [23,24] containing 403 patients using Jupyter notebook as a case study in basic exploratory data analysis [25]. The notebook (https://bit.ly/SiPy_EDA) is located in sipy/notebook folder as diabetes97.ipynb with sample screenshots of results in Figure 2. After reading the data, a heatmap based on correlation was generated using Seaborn [26] to get a sense of correlations between numerical attributes (Figure 2A). For the purpose of this case study, we shall only look at 3 attributes – waist circumference, hip circumference and high-density lipoprotein (HDL) level. The heatmap suggests high correlation between waist and hip circumferences, which has been recognized [27]; but lower correlation between HDL level to both waist and hip circumferences. We will want to examine the distributions of these 3 attributes to ensure that the data appears to be normally distributed before deciding on a correlation to use for the next step (Figure 2B). In this case, the distributions of all 3 attributes appear to be reasonably normal; hence, Pearson's correlation which is a parametric measure is suitable for the next step. A correlation test was performed between waist and hip circumferences (Figure 2C), confirming highly significant correlation between waist and hip measurements ($r = 0.83708$; 95% CI = [0.81, 0.86]; p-value = 1.45e-106); and the scatterplot suggests reasonably linearity. However, there is lesser and inverse but significant correlations between waist and HDL measurements ($r = -0.268369$; 95% CI = [-0.36, -0.17]; p-value = 5.01e-08), and hip and HDL measurements ($r = -0.21006$; 95% CI = [-0.3, -0.11]; p-value = 0.000023) but linearity cannot be commented from the scatterplots (Figure 2D). Nevertheless, these findings were consistent with the heatmap (Figure 2A). On the practical side, this example demonstrates that Seaborn [26] plots can be displayed in-line with Jupyter notebooks via SiPy kernel and statistical test can be performed together in a cell with plotting.

Discussion

SiPy adopts a deliberately conservative position on what it means to be a polyglot kernel. In the broader Jupyter ecosystem, polyglot support is most commonly realised through multi-language cell kernels, in which individual notebook cells may switch execution languages via cell magics (for example, %%python, %%r, %%julia) [28]. While this model offers apparent convenience and rapid interactivity, it also introduces a number of technical challenges [29,30]. In a multi-language cell kernel, the notebook itself becomes the primary computational environment. Execution semantics, data exchange, and state management are mediated by the kernel, often through implicit object translation or in-memory marshalling [28,31]. This can blur language boundaries, obscure provenance, and create notebooks whose behaviour is tightly coupled to a specific kernel implementation. This may result in weakened reproducibility outside the notebook environment, as the analysis cannot be straightforwardly reconstructed from command-line execution or standalone scripts. Moreover, maintaining such kernels requires continual adaptation to evolving language APIs, increasing long-term maintenance burden and conceptual complexity [32,33].

Hence, SiPy explicitly rejects this model. Instead, it treats polyglotism as a problem of coordination rather than language unification. In the coordination document model adopted by SiPy, the notebook functions as an executable record that orchestrates external scripts written in their native languages. R, Julia, Python, and shell commands are executed as first-class external artefacts, not as embedded fragments within a notebook-specific execution context [22]. This means that language boundaries are explicit, and responsibility for methodological correctness remains with the backend scripts themselves as SiPy is able to execute external scripts since version 0.8.0. As such, the design principle of SiPy kernel is closer to that of Script of Scripts [34]. This design has several practical advantages. First, it preserves reproducibility and portability [16] as any script executed via SiPy can be run independently of Jupyter,

Panel A

http://localhost:8888/lab

File Edit View Run Kernel Tabs Settings Help

jupyter_notebook.ipynb

Code SiPy

[1]:
```
%%html
<h2>This is a sample notebook on SiPy Kernel</h2>
```

This is a sample notebook on SiPy Kernel

[2]:
```
read excel surdata from data/survival_dataset.xlsx data
```

```
Read Excel: data/survival_dataset.xlsx.data into surdata
```

[3]:
```
# Change timeout from 30 seconds to 60 seconds
session.get_timeout
session.set_timeout = 60

# Check (default should be True) and set security level to True
session.get_security
session.set_security = True
```

```
SiPy Kernel execution timeout is 30 seconds
SiPy Kernel execution timeout set to 60 seconds
Security enabled: True
Security set to True
```

[4]:
```
# Attempt to run an unsafe command - should be blocked
.del random.txt
```

```
[SECURITY] Policy Violation

Command: .del random.txt

Restricted keywords detected: 'del ', '.del'

Your command contains operations that are blocked for security reasons.
Blocked keywords: file operations (open, del), imports, eval, system calls, etc.

[TIP] Use SiPy commands instead. To disable security, set:
   SIPY_SECURITY_ENABLED=false or
   session.set_security=false
```

Panel B

[5]:
```
session.get_timeout
ttest paired dataframe wide data=surdata.biomarker_baseline,surdata.biomarker_3m
ttest paired dataframe wide data=surdata.biomarker_3m,surdata.biomarker_6m
ttest paired dataframe wide data=surdata.biomarker_baseline,surdata.biomarker_6m
```

```
SiPy Kernel execution timeout is 60 seconds
                T  dof alternative          p-val         CI95%   cohen-d        BF10 power
T-test  26.192775  999   two-sided  1.530873e-115  [2.76, 3.21]  0.292818  5.046e+111   NaN
                T  dof alternative          p-val         CI95%   cohen-d        BF10     power
T-test   7.789442  999   two-sided  1.680264e-14  [0.99, 1.66]  0.121873  2.023e+11  0.970641
                T  dof alternative          p-val         CI95%   cohen-d        BF10  power
T-test  24.928151  999   two-sided  4.835367e-107  [3.97, 4.65]  0.406297  1.707e+103    1.0
```

[6]:
```
ranova ancova data=surdata y=age x=stage covariates=biomarker_baseline posthoc=lsd
```

```
Response: age
Factors: ['stage']
Covariates: ['biomarker_baseline']
Df Sum Sq Mean Sq F value Pr(>F)
stage                2     54   26.91   0.261  0.771
biomarker_baseline   1     12   12.09   0.117  0.732
Residuals          996 102818  103.23
[1] "LSD posthoc test:"
$statistics
   MSerror  Df    Mean       CV
  103.1396 997 59.1313 17.17494

$parameters
        test p.ajusted name.t ntr alpha
  Fisher-LSD      none  stage   3  0.05

$means
         age       std   r        se      LCL      UCL  Min  Max    Q25  Q50
```

Panel C

http://localhost:8888/lab

File Edit View Run Kernel Tabs Settings Help

jupyter_notebook.ipynb

Code SiPy

```
[9]: execute shell command="dir /p | findstr "sipy*""

Command to run: dir /p | findstr "sipy*"
Directory of C:\Dropbox\MyProjects\sipy
10/01/2026  12:25 am            17,093 conda_sipy_environment.txt
16/12/2025  07:27 pm    <DIR>          libsipy
10/01/2026  12:25 am             2,614 pip_sipy_environment.txt
06/02/2026  07:50 pm             5,985 sipy-kernel.log
06/02/2026  02:14 pm           224,136 sipy.py
12/01/2026  03:28 pm             2,554 sipy_info.py
11/01/2026  09:57 pm    <DIR>          sipy_kernel
04/04/2025  04:17 pm    <DIR>          sipy_plugins
04/05/2025  12:47 pm             3,837 sipy_pm.py

[10]: .dir /p | findstr "sipy*"

Command to run: dir /p | findstr "sipy*"
Directory of C:\Dropbox\MyProjects\sipy
10/01/2026  12:25 am            17,093 conda_sipy_environment.txt
16/12/2025  07:27 pm    <DIR>          libsipy
10/01/2026  12:25 am             2,614 pip_sipy_environment.txt
06/02/2026  07:50 pm             5,985 sipy-kernel.log
06/02/2026  02:14 pm           224,136 sipy.py
12/01/2026  03:28 pm             2,554 sipy_info.py
11/01/2026  09:57 pm    <DIR>          sipy_kernel
04/04/2025  04:17 pm    <DIR>          sipy_plugins
04/05/2025  12:47 pm             3,837 sipy_pm.py
```

Panel D

http://localhost:8888/lab

File Edit View Run Kernel Tabs Settings Help

jupyter_notebook.ipynb

Code SiPy

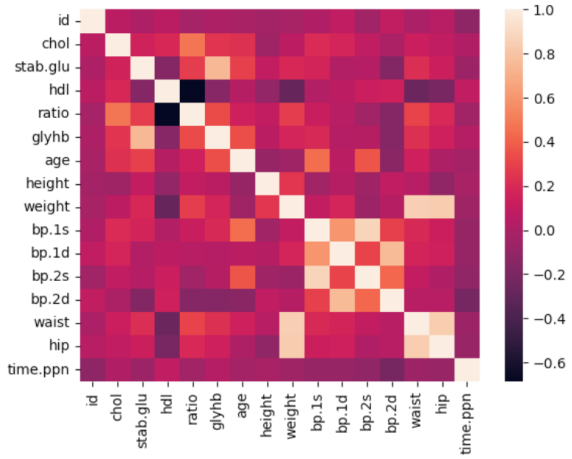
```
[13]: execute julia example_scripts\\julia_lm.jl inputfile=example_scripts\\lm_data.csv response=yN pre

Command to run: C:\Dropbox\MyProjects\sipy\portable_julia\bin\julia.exe --startup-file=no C:\Drop
box\MyProjects\sipy\example_scripts\julia_lm.jl --inputfile example_scripts\\lm_data.csv --respon
se yN --predictors "x1,x2,x3,x4,x5"
OLS Regression Results:
Response: yN
Predictors: x1, x2, x3, x4, x5
Coefficients:
[12.045793542011443, -0.023352405307506683, -0.01061719410951987, 0.01416489286203465, 0.08901173
277843084, -0.11249636916806162]

RMSE: 0.13617144047533922
R-squared: 0.9979415569097669

[14]: execute python example_scripts\\python_lm.py inputfile=example_scripts\\lm_data.csv response=yN p

Command to run: C:\Users\mauri\anaconda3\envs\sipy\python.exe C:\Dropbox\MyProjects\sipy\example_
scripts\python_lm.py --inputfile example_scripts\\lm_data.csv --response yN --predictors "x1,x2,x
3,x4,x5"
OLS Regression Results:
Response: yN
Predictors: x1, x2, x3, x4, x5
Coefficients:
[ 1.20457935e+01 -2.33524053e-02 -1.06171941e-02  1.41648929e-02
  8.90117328e-02 -1.12496369e-01]

RMSE: 0.13617144047533947
R-squared: 0.9979415569097669
```

Figure 1: Screenshots of Sample Juypter Notebook. Panel A shows a HTML cell, reading of an Excel file into SiPy, changing session parameters, and attempting to execute an unsafe command. Panel B shows execution of t-tests, and R-based ANOVA. Panel C shows the execution of commands. Panel D shows execution of Julia script, and Python script.

Panel A

```
[2]: # generate a heatmap of correlations from the data
     plot heatmap dataframe data=db
```

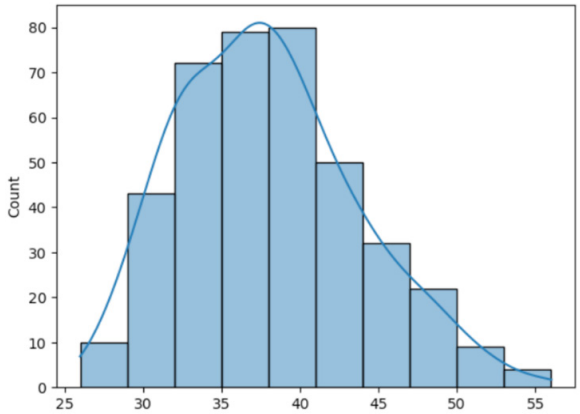
```
Regplot plotted successfully for dataframe 'db'
```



Panel B

```
[3]: plot histogram dataframe data=db.waist bins=10 kde=true
     plot histogram dataframe data=db.hip bins=10 kde=true
     plot histogram dataframe data=db.hdl bins=10 kde=true
```

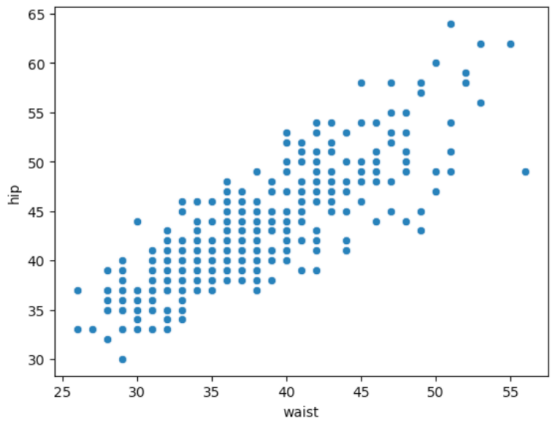
```
Histogram plotted successfully for column 'waist' in dataframe 'db'
Histogram plotted successfully for column 'hip' in dataframe 'db'
Histogram plotted successfully for column 'hdl' in dataframe 'db'
```



Panel C

```
[4]: # do a correlation test between waist and hip measurements, and generate a scatterplot
     correlate pearson dataframe wide data=db.waist,db.hip
     plot scatterplot dataframe data=db x=waist y=hip
```

```
           n        r         CI95%          p-val        BF10  power
pearson  401  0.83708  [0.81, 0.86]  1.446854e-106  6.213e+102    1.0
Scatterplot plotted successfully for dataframe 'db'
```



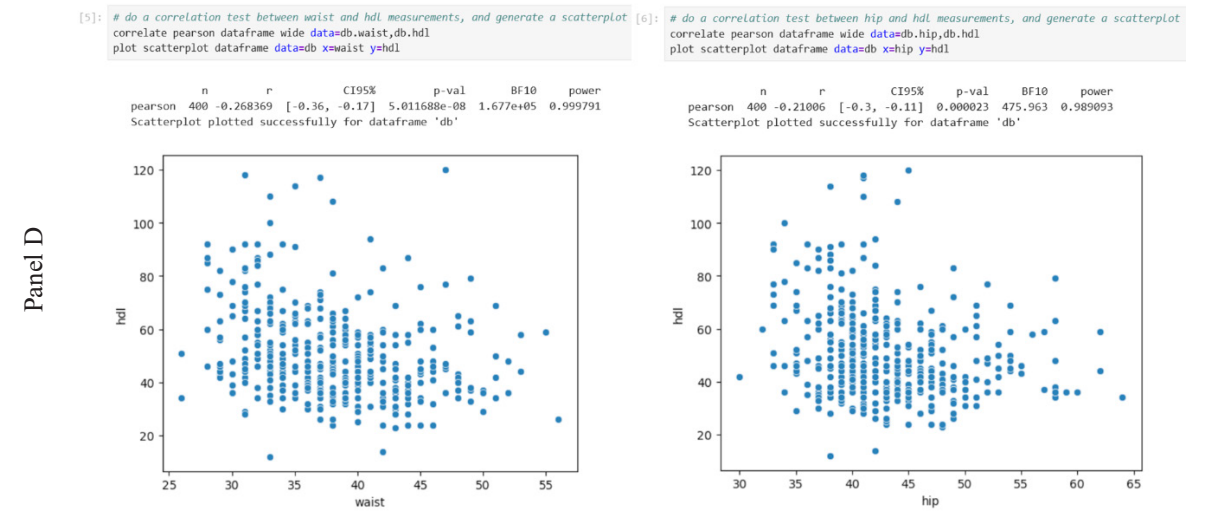


Figure 2: Screenshots of Sample Exploratory Data Analysis. Panel A shows a heatmap of correlations on numerical attributes. Panel B shows the distribution of waist circumference, which appears to be reasonably normally distributed. The distributions of hip circumference and HDL level are in the notebook. Panel C show the correlation between waist and hip circumferences. Panel D shows the correlations between waist and hip circumferences, to HDL levels.

using standard command-line tools. Second, it respects disciplinary expertise by allowing users to work directly within the idioms, libraries, and workflows of their chosen language, without adaptation to a notebook-centric abstraction. Third, it keeps the SiPy kernel lightweight and stable [35], as extensibility is achieved through script execution rather than function-level wrapping or language emulation.

However, the coordination document model does impose certain constraints. It sacrifices some immediacy of interactive, cross-language data manipulation, and it places greater responsibility on users to manage inputs and outputs explicitly. Yet, these constraints are intentional. SiPy prioritises transparency over convenience, explicit execution over implicit state, and long-term maintainability over short-term feature accumulation. In this sense, the notebook is not treated as a computational substrate but as a structured execution narrative that records how heterogeneous tools are composed into a coherent analytical workflow. By adopting this stance, SiPy positions itself not as a competitor to multi-language notebook systems but as an alternative architectural response to polyglot scientific computing. It functions as a reproducible execution scaffold rather than a language abstraction layer. This choice aligns with SiPy's broader design philosophy: to remain minimal, explicit, and methodologically honest, while enabling complex, multi-language analyses to coexist within a single, inspectable analytical document.

Conclusion

In this study, we present SiPy kernel not as a polyglot interpreter but as a coordination and documentation interface for script-based scientific workflows. By preserving explicit language boundaries and delegating computation to standalone back-end scripts, it supports reproducible, portable, and maintainable multi-language analysis while remaining lightweight and methodologically transparent.

Data Availability

Source codes SiPy can be found at https://github.com/mauriceling/sipy with the implementation of SiPy Kernel at https://github.com/mauriceling/sipy/tree/main/sipy_kernel. A sample notebook can be found at https://bit.ly/SiPy_Notebook_Sample and the basic exploratory data analysis notebook can be found at https://bit.ly/SiPy_EDA.

References

1. Perez F, Granger BE IPython: A System for Interactive Scientific Computing. Computing in Science & Engineering. 2007; 9(3): 21-29. DOI: 10.1109/MCSE.2007.53.
2. Perkel JM. Ten Computer Codes That Transformed Science. Nature. 2021; 589(7842): 344-348. DOI: 10.1038/d41586-021-00075-2.
3. Perkel JM. Why Jupyter is Data Scientists' Computational Notebook of Choice. Nature. 2018; 563(7729): 145-146. DOI: 10.1038/d41586-018-07196-1.
4. Pimentel JF, Murta L, Braganholo V, Freire J. A Large-Scale Study About Quality and Reproducibility of Jupyter Notebooks. IEEE/ACM. 2019; 507-517. DOI: 10.1109/MSR.2019.00077.
5. Granger BE, Perez F. Jupyter: Thinking and Storytelling With Code and Data. Computing in Science & Engineering. 2021; 23(2):7-14. DOI: 10.1109/MCSE.2021.3059263.
6. Siddik MS, Li H, Bezemer C-P. A Systematic Literature Review of Software Engineering Research on Jupyter Notebook. Journal of Systems and Software. 2026; 235: 112758. DOI: 10.1016/j.jss.2025.112758.
7. Wang AY, Wang D, Drozdal J, Liu X, Park S, et al. What Makes a Well-Documented Notebook? A Case Study of Data Scientists' Documentation Practices in Kaggle. ACM. 2021: 1-7. DOI: 10.1145/3411763.3451617.
8. Chen E, Asta M. Using Jupyter Tools to Design an Interactive Textbook to Guide Undergraduate Research in Materials Informatics. Journal of Chemical Education. 2022; 99(10): 3601-3606. DOI: 10.1021/acs.jchemed.2c00640.
9. Yavuz Temel G, Barenthien J, Padubrin T. Using Jupyter Notebooks as Digital Assessment Tools: An Empirical Examination of Student Teachers' Attitudes and Skills Towards Digital Assessment.

Education and Information Technologies. 2025; 30(13): 18621-18650. https://doi.org/10.1007/s10639-025-13507-7

10. Bascuñana J, León S, González-Miquel M, González EJ, Ramírez J. Impact of Jupyter Notebook as a Tool to Enhance the Learning Process in Chemical Engineering Modules. Education for Chemical Engineers. 2023; 44:155-163. DOI: 10.1016/j.ece.2023.06.001.
11. Verrett J, Boukouvala F, Dowling A, Ulissi Z, Zavala V. Computational Notebooks in Chemical Engineering Curricula. Chemical Engineering Education. 2020; 54(3). DOI: 10.18260/2-1-370.660-116661.
12. Kluyver T, Ragan-Kelley B, Perez F, Granger B, Bussonnier M, et al. Jupyter Notebooks - A Publishing Format for Reproducible Computational Workflows. Positioning and Power in Academic Publishing: Players, Agents and Agendas (IOS Press). 2016: 87-90. DOI:10.3233/978-1-61499-649-1-87.
13. Beg M, Taka J, Kluyver T, Konovalov A, Ragan-Kelley M, et al. Using Jupyter for Reproducible Scientific Workflows. Computing in Science & Engineering. 2021; 23(2): 36-46. DOI: 10.1109/MCSE.2021.3052101.
14. Huang RL, Ravi S, He M, Tian B, Lerner S, et al. How Scientists Use Jupyter Notebooks: Goals, Quality Attributes, and Opportunities. IEEE/ACM. 2025: 1243-1255. DOI: 10.1109/ICSE55347.2025.00232.
15. Wang J, Kuo T, Li L, Zeller A. Assessing and Restoring Reproducibility of Jupyter Notebooks. IEEE/ACM 2020: 138-149. DOI: 10.1145/3324884.3416585.
16. Samuel S, Mietchen D. Computational Reproducibility of Jupyter Notebooks from Biomedical Publications. Gigascience. 2024: 13: giad113. DOI: 10.1093/gigascience/giad113.
17. Ramasamy D, Sarasua C, Bacchelli A, Bernstein A. Workflow Analysis of Data Science Code in Public GitHub Repositories. Empirical Software Engineering. 2023; 28(1): 7. DOI: 10.1007/s10664-022-10229-z.
18. Tan NT, Mugundhan M, Liu T, Tan RY, Tang AY, et al. SiPy - Bringing Python and R to the End-User in a Plugin-Extensible System. Medicon Medical Sciences. 2025; 8(6): 32-41. DOI: 10.55162/MCMS.08.295.
19. Ambel WB, Ling MH. SiPy 0.7.0 – R-Based ANOVA and Survival Analyses. Open Access J Sci. 2026; 9: 1-5. DOI: 10.15406/oajs.2026.09.00280.
20. Colonnelli I, Aldinucci M, Cantalupo B, Padovani L, Rabellino S, et al. Distributed Workflows with Jupyter. Future Generation Computer Systems. 2022; 128: 282-298. DOI: 10.1016/j.future.2021.10.007.
21. Kumar A, Cuccuru G, Grüning B, Backofen R. An Accessible Infrastructure for Artificial Intelligence Using a Docker-Based JupyterLab in Galaxy. GigaScience. 2022; 12: giad028. DOI: 10.1093/gigascience/giad028.
22. Ling MH. SiPy 0.8.0 on the Three Legs of Python, R, and Julia. Acta Scientific Computer. 2026; 8(1). DOI: 10.31080/ASCS.2026.08.0596.
23. Willems JP, Saunders JT, Hunt DE, Schorling JB. Prevalence of coronary heart disease risk factors among rural blacks: a community-based study. Southern Medical Journal. 1997; 90(8): 814-820. DOI: 10.1097/00007611-199708000-00008.
24. Schorling JB, Roach J, Siegel M, Baturka N, Hunt DE, et al. A trial of church-based smoking cessation interventions for rural African Americans. Prev Med. 1997; 26(1): 92-101. DOI: 10.1006/pmed.1996.9988.
25. Tukey JW. Exploratory Data Analysis (Addison-Wesley Publishing Company). 1977. https://books.google.com.sg/books?id=UT9dAAAAIAAJ
26. Waskom M. Seaborn: Statistical Data Visualization. Journal of Open Source Software. 2021; 6(60): 3021. DOI: 10.21105/joss.03021.
27. Snijder MB, Dekker JM, Visser M, Bouter LM, Stehouwer CD, et al. Associations of hip and thigh circumferences independent of waist circumference with the incidence of type 2 diabetes: the Hoorn Study. Am J Clin Nutr. 2003; 77(5): 1192-7. DOI: 10.1093/ajcn/77.5.1192.
28. Peng B, Wang G, Ma J, Leong MC, Wakefield C, et al. SoS Notebook: An Interactive Multi-Language Data Analysis Environment. Bioinformatics. 2018; 34(21): 3768-3770. DOI: 10.1093/bioinformatics/bty405.
29. Mussbacher G, Combemale B, Kienzle J, Burgueño L, Garcia-Dominguez A, et al. Polyglot Software Development: Wait, What? IEEE Software. 2024; 41(4):124-133. DOI: 10.1109/MS.2023.3347875.
30. Yang H, Nong Y, Wang S, Cai H. Multi-Language Software Development: Issues, Challenges, and Solutions. IEEE Transactions on Software Engineering. 2024; 50(3): 512-533. DOI: 10.1109/TSE.2024.3358258.
31. Niephaus F, Krebs E, Flach C, Lincke J, Hirschfeld R. PolyJuS: a Squeak/Smalltalk-based polyglot notebook system for the GraalVM. ACM: 2019: 1-6. DOI: 10.1145/3328433.3328434.
32. Houdaille P, Khelladi DE, Briend R, Jongeling R, Combemale B. Polyglot AST: Towards Enabling Polyglot Code Analysis. (ICECCS) (IEEE, Toulouse). 2023: 116-125. DOI: 10.1109/ICECCS59891.2023.00023.
33. Li Z, Qi X, Yu Q, Liang P, Mo R, et al. Exploring Multi-Programming-Language Commits and Their Impacts on Software Quality: An Empirical Study on Apache Projects. Journal of Systems and Software. 2022; 194: 111508. DOI: 10.1016/j.jss.2022.111508.
34. 34.Wang G, Peng B. Script of Scripts: A pragmatic workflow system for daily computational research. PLoS Comput Biol. 2019; 15(2): e1006843. DOI: 10.1371/journal.pcbi.1006843.
35. 35. Mernik M, Zumer V. Domain-specific languages for software engineering. IEEE Comput. Soc, Maui, HI. 2001: 4002-4002. DOI: 10.1109/HICSS.2001.927266.